

Escola Superior de Tecnologia e Gestão de Águeda

Sistemas Operativos

Técnico Superior Profissional em Redes e Sistemas Informáticos

Introdução ao Python

Professor: Ivan Miguel Pires

2025/2026

OBJETIVOS DE APRENDIZAGEM

- Conhecer a linguagem Python e suas características
- Dominar conceitos fundamentais: variáveis, tipos, operadores
- Utilizar estruturas de controlo (if, for, while)
- Trabalhar com estruturas de dados (listas, dicionários)
- Criar e utilizar funções
- Manipular ficheiros
- Conhecer boas práticas modernas (2024-2026)
- Explorar ferramentas e frameworks modernos
- Compreender o ecossistema Python atual

O QUE É PYTHON?

- Linguagem de programação criada por Guido van Rossum
 - Final dos anos 1980 (CWI, Holanda)
 - Primeira versão: 1991
- Características principais:
 - Interpretada (não necessita compilação)
 - Tipagem dinâmica
 - Sintaxe simples e legível
 - Indentação obrigatória (blocos de código)
 - Multi-paradigma (procedural, OO, funcional)
- Versões atuais (2024-2026):
 - Python 3.12 (outubro 2023)
 - Python 3.13 (outubro 2024)

PORQUÊ PYTHON?

- Popularidade:
 - Uma das linguagens mais populares do mundo
 - Grande comunidade e ecossistema
- Versatilidade:
 - Web development (Django, Flask, FastAPI)
 - Data Science e ML (pandas, scikit-learn, PyTorch)
 - Automação e scripting
 - Cloud e DevOps
 - IoT e sistemas embebidos
- Facilidade de aprendizagem:
 - Sintaxe próxima da linguagem natural
 - Curva de aprendizagem suave

INSTALAÇÃO E AMBIENTES DE DESENVOLVIMENTO

- Instalação:
 - python.org/downloads
 - Versão recomendada: Python 3.12+
- Ambientes de desenvolvimento:
 - IDLE (incluído com Python)
 - VS Code (mais popular, extensões Python)
 - PyCharm (IDE completo)
 - Jupyter Notebook (Data Science)
 - Google Colab (cloud, gratuito)
- Terminal/Shell:
 - `python` ou `python3` (executar interpretador)
 - `python script.py` (executar ficheiro)

VARIÁVEIS E TIPOS DE DADOS

- Variáveis (sem declaração de tipo):
 - nome = 'João'
 - idade = 25
 - altura = 1.75
- Tipos básicos:
 - int - inteiros (42, -10)
 - float - reais (3.14, -0.5)
 - str - strings ('texto', "texto")
 - bool - booleanos (True, False)
- Verificar tipo: `type(variavel)`
- Conversão: `int()`, `float()`, `str()`
- Python 3.12+: Melhor performance em operações

MANIPULAÇÃO DE STRINGS

- Operações básicas:
 - Concatenação: 'Olá' + ' ' + 'Mundo'
 - Repetição: 'Ha' * 3 → 'HaHaHa'
 - Tamanho: len('Python') → 6
- Indexação e slicing:
 - texto[0] - primeiro caracter
 - texto[-1] - último caracter
 - texto[0:5] - substring
- Métodos úteis:
 - .upper(), .lower(), .strip()
 - .split(), .replace(), .find()
- F-strings (moderno):
 - f'Nome: {nome}, Idade: {idade}'

OPERADORES EM PYTHON

- Aritméticos:
 - +, -, *, / (divisão real)
 - // (divisão inteira), % (resto)
 - ** (potência)
- Comparação:
 - ==, !=, <, >, <=, >=
- Lógicos:
 - and, or, not
- Atribuição:
 - =, +=, -=, *=, /=
- Pertença:
 - in, not in (verificar em listas/strings)

ESTRUTURA CONDICIONAL: IF

- Sintaxe básica:
 if condição:
 # código se verdadeiro
- if...else:
 if condição:
 # código se verdadeiro
 else:
 # código se falso
- if...elif...else:
 if condição1:
 # código 1
 elif condição2:
 # código 2
 else:
 # código padrão
- Importante: Indentação obrigatória!

CICLO DE REPETIÇÃO: FOR

- Iterar sobre sequências:
 for item in lista:
 print(item)
- Função range():
 - range(5) → 0, 1, 2, 3, 4
 - range(1, 6) → 1, 2, 3, 4, 5
 - range(0, 10, 2) → 0, 2, 4, 6, 8
- Exemplo:
 for i in range(5):
 print(i)
- Iterar com índice:
 for i, valor in enumerate(lista):
 print(f'Índice {i}: {valor}')

CICLO DE REPETIÇÃO: WHILE

- Sintaxe:

```
while condição:  
    # código a repetir
```

- Exemplo:

```
contador = 0  
while contador < 5:  
    print(contador)  
    contador += 1
```

- Controlo de ciclos:

break - sair do ciclo

continue - saltar para próxima iteração

- Cuidado: Evitar ciclos infinitos!

LISTAS (ARRAYS)

- Criação:
 - `numeros = [1, 2, 3, 4, 5]`
 - `nomes = ['Ana', 'Bruno', 'Carlos']`
 - `mista = [1, 'texto', 3.14, True]`
- Acesso:
 - `lista[0]` - primeiro elemento
 - `lista[-1]` - último elemento
 - `lista[1:4]` - slicing
- Métodos principais:
 - `.append(x)` - adicionar ao fim
 - `.insert(i, x)` - inserir em posição
 - `.remove(x)` - remover elemento
 - `.sort()` - ordenar
 - `.reverse()` - inverter

LIST COMPREHENSIONS

- Forma concisa de criar listas:
- Sintaxe:
 - [expressão for item in iterável]
- Exemplos:
 - quadrados = [x**2 for x in range(10)]
 - pares = [x for x in range(20) if x % 2 == 0]
 - maiusculas = [s.upper() for s in nomes]
- Com condição:
 - [expressão for item in iterável if condição]
- Vantagens:
 - Código mais conciso e legível
 - Geralmente mais rápido que loops

DICIONÁRIOS (CHAVE:VALOR)

- Criação:
 - `pessoa = {'nome': 'João', 'idade': 25}`
 - `vazio = {}` ou `dict()`
- Acesso:
 - `pessoa['nome'] → 'João'`
 - `pessoa.get('idade') → 25`
- Adicionar/Modificar:
 - `pessoa['cidade'] = 'Lisboa'`
- Métodos úteis:
 - `.keys()` - todas as chaves
 - `.values()` - todos os valores
 - `.items()` - pares chave-valor
- Iterar:
 - `for chave, valor in dicionario.items():`

FUNÇÕES EM PYTHON

- Definição:

```
def nome_funcao(parametros):  
    # código  
    return resultado
```

- Exemplo:

```
def soma(a, b):  
    return a + b
```

- Chamada:

- resultado = soma(5, 3)

- Parâmetros opcionais:

```
def saudar(nome, saudacao='Olá'):  
    return f'{saudacao}, {nome}!'
```

- Docstrings (documentação):

- `"""Descrição da função"""`

TYPE HINTS: ANOTAÇÕES DE TIPO

- Python 3.5+: Anotações opcionais de tipo
- Sintaxe:
 - `def soma(a: int, b: int) -> int:`
 - `return a + b`
- Variáveis com tipo:
 - `nome: str = 'João'`
 - `idade: int = 25`
- Tipos complexos:
 - `from typing import List, Dict, Optional`
 - `def processar(dados: List[int]) -> Dict[str, int]:`
- Ferramentas de verificação:
 - mypy - verificador de tipos estático
 - pyright - rápido, usado no VS Code
- Benefícios: Melhor IDE support, menos bugs

MANIPULAÇÃO DE FICHEIROS

- Abrir ficheiro:
 - `f = open('ficheiro.txt', 'r')` # leitura
 - `f = open('ficheiro.txt', 'w')` # escrita
 - `f = open('ficheiro.txt', 'a')` # append
- Ler:
 - `conteudo = f.read()` # tudo
 - `linha = f.readline()` # uma linha
 - `linhas = f.readlines()` # lista de linhas
- Escrever:
 - `f.write('texto\n')`
- Fechar:
 - `f.close()`
- Moderno (with statement):
 - `with open('ficheiro.txt', 'r') as f:`
 - `conteudo = f.read()`

MÓDULOS E BIBLIOTECAS

- Importar módulos:
 - `import math`
 - `from math import sqrt, pi`
 - `import numpy as np # alias`
- Módulos da biblioteca padrão:
 - `math` - funções matemáticas
 - `random` - números aleatórios
 - `datetime` - datas e horas
 - `os` - interação com SO
 - `json` - trabalhar com JSON
- Instalar pacotes externos:
 - `pip install nome_pacote`
- Exemplo:
 - `import math`
 - `print(math.sqrt(16)) # 4.0`

BOAS PRÁTICAS PYTHON (2024-2026)

- Estilo de código:
 - PEP 8 - guia de estilo oficial
 - black - formatador automático
 - ruff - linter moderno e rápido
- Type hints:
 - Usar anotações de tipo
 - mypy ou pyright para verificação
- Testes:
 - pytest - framework de testes moderno
- Ambientes virtuais:
 - venv ou virtualenv
 - Isolar dependências por projeto
- Gestão de dependências:
 - requirements.txt ou pyproject.toml
 - poetry ou uv (modernos)

FERRAMENTAS DO ECOSSISTEMA PYTHON

- Formatação e Linting:
 - black - formatador (opinionated)
 - ruff - linter rápido (Rust)
 - pylint - análise estática
- Type Checking:
 - mypy - verificador de tipos
 - pyright - rápido, Microsoft
- Testes:
 - pytest - framework moderno
 - coverage.py - cobertura de testes
- Gestão de Projetos:
 - poetry - dependências e packaging
 - uv - gerenciador ultra-rápido (2024)
- Documentação:
 - Sphinx - gerar documentação

FastAPI: APIs Modernas em Python

- O que é:
 - Framework web moderno e rápido
 - Baseado em type hints
 - Async/await nativo
- Características:
 - Alta performance (comparável a Node.js)
 - Documentação automática (OpenAPI)
 - Validação automática (Pydantic)
- Exemplo básico:

```
from fastapi import FastAPI
app = FastAPI()
@app.get('/')
def read_root():
    return {'Hello': 'World'}
```
- Casos de uso: APIs REST, ML inference

PYTHON PARA DATA SCIENCE E ML

- Bibliotecas essenciais:
 - NumPy - arrays e operações numéricas
 - pandas - manipulação de dados
 - matplotlib/seaborn - visualização
 - scikit-learn - machine learning
- Deep Learning:
 - PyTorch - framework ML moderno
 - TensorFlow - framework Google
- Ferramentas:
 - Jupyter Notebook - ambiente interativo
 - Google Colab - cloud gratuito
- Tendências 2024-2026:
 - PyTorch dominante em research
 - Reprodutibilidade e MLOps

PROGRAMAÇÃO ASSÍNCRONA

- `async/await` (Python 3.5+):
 - `async def fetch_data():`
 - `await asyncio.sleep(1)`
 - `return 'dados'`
- Benefícios:
 - I/O não bloqueante
 - Melhor performance em I/O-bound
 - Ideal para APIs, web scraping
- Performance (2024-2026):
 - Até 3.1× melhoria em concorrência
 - `asyncio` melhorado em Python 3.12+
- Quando usar:
 - Muitas operações I/O
 - Requests HTTP simultâneos
 - Websockets, streaming

PYTHON NA CLOUD

- Serverless:
 - AWS Lambda - funções serverless
 - Azure Functions - Microsoft
 - Google Cloud Functions
- Otimizações (2024-2026):
 - Cold start: 2.26× mais rápido
 - Memória: 1.51× redução
 - Startup LLMs: 6-8× mais rápido
- Containers:
 - Docker - containerização
 - Kubernetes - orquestração
- Boas práticas:
 - Minimizar dependências
 - Pré-carregar modelos ML
 - Usar camadas Lambda

SEGURANÇA E BOAS PRÁTICAS

- Vulnerabilidades comuns:
 - Injeção SQL (usar ORMs, prepared statements)
 - Deserialização insegura (evitar pickle)
 - Dependências vulneráveis
- Ferramentas de segurança:
 - bandit - scanner de segurança
 - safety - verificar dependências
 - pip-audit - auditoria de pacotes
- Boas práticas:
 - Nunca hardcode secrets
 - Usar variáveis de ambiente
 - Validar inputs (Pydantic)
 - Atualizar dependências regularmente
- Gestão de secrets: python-dotenv, Vault

AMBIENTES VIRTUAIS

- Porquê usar:
 - Isolar dependências por projeto
 - Evitar conflitos de versões
 - Reprodutibilidade
- Criar ambiente virtual:
 - `python -m venv venv`
- Ativar:
 - Windows: `venv\Scripts\activate`
 - Linux/Mac: `source venv/bin/activate`
- Instalar pacotes:
 - `pip install pacote`
- Exportar dependências:
 - `pip freeze > requirements.txt`
- Instalar de ficheiro:
 - `pip install -r requirements.txt`
- Moderno: poetry, uv para gestão completa

PYTHON NO MUNDO REAL (2024-2026)

- Web Development:
 - Instagram, Spotify, Reddit (Django)
 - APIs modernas (FastAPI)
- Data Science e AI:
 - Análise de dados (pandas, NumPy)
 - Machine Learning (scikit-learn, PyTorch)
 - LLMs e AI (OpenAI, Hugging Face)
- DevOps e Automação:
 - Ansible - automação de infraestrutura
 - Scripts de deployment
- Cloud e Serverless:
 - AWS Lambda, Azure Functions
 - Microservices
- Ciência:
 - Astronomia, Bioinformática, Física

CONCLUSÕES

- Python é versátil e poderoso
- Sintaxe simples, fácil de aprender
- Ecossistema rico:
 - Bibliotecas para tudo
 - Comunidade ativa e grande
- Moderno e em evolução:
 - Python 3.12/3.13 com melhorias
 - Ferramentas modernas (FastAPI, ruff, uv)
 - Type hints cada vez mais importantes
- Aplicações em todas as áreas:
 - Web, Data Science, ML, Cloud, DevOps
- Aprendizagem contínua essencial
- Prática é fundamental!

QUESTÕES?